WEEK 1 / LESSON 3 / 60 MINUTES

L3

Python Programming Essentials

The mental model behind 90% of production AI code. Variables as labels, six core types, the early-return pattern, and Pythonic loops — taught from the inside out, so the language stops surprising you and starts obeying you.

WHAT'S INSIDE

WHY PYTHON

90% of production AI is Python — for one quiet reason

PRODUCTION AI

90%

Of production AI workloads run on Python somewhere in the stack.

DATA STRUCTURES

6

Core types are enough to build almost anything you need.

NAMES

labels

Not boxes — once you see this, the language obeys you.

READABILITY

5x

Pythonic loops are roughly five times shorter than C-style ones.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1. Declare and use Python's six core data types confidently.
2. Explain how dynamic + strong typing actually works.
3. Pick the right data structure for any small problem.
4. Write conditionals using the early-return pattern.
5. Convert C-style loops into Pythonic iteration.
6. Use list, dict, and set comprehensions correctly.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01
The object model

Why every variable is a label, and every value is an object — the mental shift that fixes everything.

02
Types & structures

Six core types, the difference between mutable and immutable, and how to choose between list, dict and set.

03
Early-return logic

Senior code avoids long if/elif chains. We'll learn the pattern that replaces them.

04
Pythonic loops

Iterate over the *thing*, not the index. Comprehensions, enumerate, zip — and when each one is the right call.

01

PART 1 OF 4

The object model — labels, not boxes

The single mental shift that makes Python feel native, not foreign.

CORE IDEA

Variables are luggage tags, not storage boxes

In C or Java, a variable is a labelled box that holds a value. Change the value, you change what's in the box.

In Python, a variable is a luggage tag stuck onto an object. The object lives somewhere; the tag just points to it. Multiple tags can point to the same suitcase. Moving a tag does not move the suitcase.

Once you see this, half the surprises in Python disappear.

OLD MENTAL MODEL

x = 7 (the box)

x is a box that holds 7. y = x copies the contents into a new box. They are independent.

PYTHON ACTUALLY

x = 7 (the tag)

x is a tag stuck onto the integer 7. y = x adds a second tag onto the *same* 7. Both names see one object.

DEMO · IT'S ALL LABELS

A two-line demo that explains 80% of beginner confusion

Run these lines in your head before you run them in Python. Then run them, and notice that you were right.

>>> PYTHON

```
x = 7
y = x        # y now also tags 7
x = 8        # x re-tags 8.  y still tags 7

print(x)     # 8
print(y)     # 7   (it never changed)
```

→ *Names are labels. Labels move. Objects don't.*

02

PART 2 OF 4

Six types, three structures, one pattern

The six data types you'll touch every day, and how to pick between list, dict and set in 10 seconds.

THE SIX CORE TYPES

Master these six and 90% of Python is yours

Memorise the example column. Each one is short on purpose — that is exactly how a senior engineer thinks about types.

#

INT

Whole numbers

x = 7 · age = 24 · count = 0

Unlimited size in Python — no overflow.

.

FLOAT

Decimal numbers

price = 9.99 · pi = 3.1416

64-bit double under the hood. Beware == on floats.

"

STR

Text

name = "Anita" · city = 'Pune'

Immutable — every edit creates a new object.

✓

BOOL

True / False

is_open = True · alive = False

Technically subclass of int — True == 1.

[]

LIST

Ordered, mutable

nums = [1, 2, 3] · names = []

Index it, append, sort, slice.

{}

DICT

Key-value map

u = {'name': 'Ravi', 'age': 24}

O(1) lookup by key. Workhorse of real Python.

MUTABILITY MATTERS

Mutable vs. immutable — the bug-source most beginners miss

Some types can be changed in place. Others can't. The difference quietly decides what your code does when two names point at the same object.

Lists, dicts, sets — mutable. You can edit them.

int, float, str, tuple, frozenset — immutable. Any 'change' actually creates a brand new object.

WHEN IT BITES

- Default arguments: never use def f(x=[]).
- Sharing a list across two names — edits one, breaks the other.
- Hashable keys: only immutable types can be dict keys.
- Equality vs identity: == is value, is is the same object.

QUICK REFERENCE

List, dict or set — pick in ten seconds

Most real Python is one of these three. Reach for them in the order shown — only fall back when the simpler one fails.

Pick	When	Lookup	Order
LIST	You need order, duplicates allowed	O(n)	Insertion order kept
DICT	You need fast lookup by a key	O(1)	Insertion order kept (3.7+)
SET	You only need uniqueness or membership	O(1)	Unordered
TUPLE	You need a fixed, immutable record	O(n)	Insertion order kept

03

PART 3 OF 4

The early-return pattern — exit clearly, exit early

The single biggest readability win in real Python code.

THE PATTERN

Senior Python returns early. Beginner Python nests forever.

Same logic, two styles. The right-hand version is what you'll see in every well-reviewed pull request.

>>> PYTHON

```python
# Beginner: nested if/elif/else
def role(user):
    if user.active:
        if user.is_admin:
            return 'admin'
        else:
            return 'member'
    else:
        return 'inactive'

# Senior: guard clauses + early return
def role(user):
    if not user.active:  return 'inactive'
    if user.is_admin:    return 'admin'
    return 'member'
```

→ *Read top-to-bottom. No mental stack to maintain. No final 'else'.*

04

PART 4 OF 4

Pythonic loops — iterate over the thing

The looping habits that separate readable code from C-with-Python-syntax.

LOOP IDIOMS

Three patterns to memorise — they cover almost everything

enumerate when you need the index, zip for parallel iteration, and a list comprehension for the simple transform.

>>> PYTHON

```
names  = ['Anita', 'Ravi', 'Priya']
scores = [88, 76, 91]

# 1. enumerate — index AND value
for i, n in enumerate(names):
    print(i, n)

# 2. zip — two lists in lockstep
for n, s in zip(names, scores):
    print(n, '→', s)

# 3. comprehension — transform & filter
passing = [n for n, s in zip(names, scores) if s >= 80]
```

→ *If you need range() and an index var, look again — there is usually a better way.*

✦ THINK · PAIR · SHARE

ACTIVITY

Spot the loop — refactor it on paper

PROMPT

Below is real-world C-style Python. Rewrite it the Pythonic way in your notebook — one line if you can.

```
total = 0
for i in range(len(prices)):
    if prices[i] > 100:
        total += prices[i]
```

HOW TO RUN IT

1. 60s solo — write the rewrite in your notebook.
2. 60s with a partner — compare the two versions.
3. 60s share-out — one team reads its version aloud.
4. 30s reveal — instructor shows the canonical answer.

WORKED EXAMPLE

Cleaning a CSV of student scores — the Pythonic walkthrough

THE SCENARIO

You receive a list of 250 student records as dicts: each has a 'name', a list of scores, and a flag for 'active'. You need the average score for active students whose average is at least 75. A junior engineer writes 18 lines with three for loops and a boolean accumulator.

THE TAKEAWAY

The Pythonic version is three lines: a comprehension to compute averages, a filter to keep the qualifying ones, then sum/len for the headline number — and it is faster too.

WHAT TO NOTICE

1. Use a comprehension when the loop is just a transform + filter.
2. Iterate over the dicts directly — never over indices.
3. Use sum() / len() over an explicit accumulator.
4. If a loop crosses 7 lines, pause — usually it can be flatter.

KNOWLEDGE CHECK · QUESTION 1 OF 3

x = [1, 2, 3] · y = x · x.append(4). What is y?

A [1, 2, 3] — y is a copy, unaffected by changes to x

B [1, 2, 3, 4] — y and x are two names for the same list

C Raises TypeError — you can't append to a list assigned to two names

D [4] — only the latest change survives

ANSWER · QUESTION 1 OF 3

x = [1, 2, 3] · y = x · x.append(4). What is y?

B

CORRECT ANSWER

[1, 2, 3, 4] — y and x tag the same list

WHY

Assignment never copies in Python. y = x just adds a second luggage tag to the existing list. Mutating the list through one name shows up through the other. To get an independent copy use list(x) or x[:].

KNOWLEDGE CHECK · QUESTION 2 OF 3

You need to look up a user's role 100,000 times by their email address. Which structure is right?

A. List of (email, role) tuples — straightforward and ordered

B. Set of email strings — guarantees uniqueness

C. Dict mapping email → role — O(1) lookup

D. Tuple of dicts — immutable for safety

ANSWER · QUESTION 2 OF 3

You need to look up a user's role 100,000 times by their email address. Which structure is right?

C

CORRECT ANSWER

Dict mapping email → role

WHY

Lookup-by-key is the canonical use case for a dict — O(1) per call. The list version is O(n) per lookup, which makes 100,000 calls explode by a factor of (n × 100,000). The set has no values to return; the tuple-of-dicts loses the speed advantage.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1 *Which type — list, dict or set — do you reach for too often? Which one too rarely?*

Q2 *Look at any function in your last code: would the early-return pattern make it shorter?*

Q3 *Find one for-loop you wrote this week and rewrite it as a comprehension. Did it get clearer or murkier?*

APPLY IT → ASSIGNMENT

Refactor a 30-line script into Pythonic code — keep behaviour identical

Take any 30-line Python script you've written before (or one we'll share). Refactor it without changing what it does — apply early returns, eliminate range(len()), and replace at least one loop with a comprehension. Submit a before-and-after diff.

DELIVERABLES

- Original script + refactored script in two files
- Diff comment summarising every change in one line each
- Before/after line count — refactored should be at most 60% of the original
- Proof the behaviour is identical — same inputs, same outputs
- One paragraph: which change made the biggest readability gain?

WHAT 'GOOD' LOOKS LIKE

A great submission is the same script — only shorter, flatter and more honest about what it actually does. Behaviour identical. Cognitive load halved.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

1. Python names are labels — they never own the object they point to.
2. Six core types do almost everything: int, float, str, bool, list, dict.
3. Mutability decides what happens when two names point at the same thing.
4. Early returns beat nested if/else for everything except trivial branches.
5. Iterate over the thing, not the index — and reach for comprehensions when the loop is just a transform.

END OF LESSON 3

What you can now do

Each one of these is now part of your working vocabulary.

1. Reach for the right one of Python's six core types without thinking.
2. Predict whether two names share state before you run the code.
3. Pick list, dict or set in under ten seconds for any small problem.
4. Replace nested conditionals with the early-return pattern on demand.
5. Convert any C-style loop to its Pythonic equivalent in one rewrite.

NEXT → LESSON 4 — Functions, Modules & Clean Code